UVA DATA SCIENCE

Midterm Review

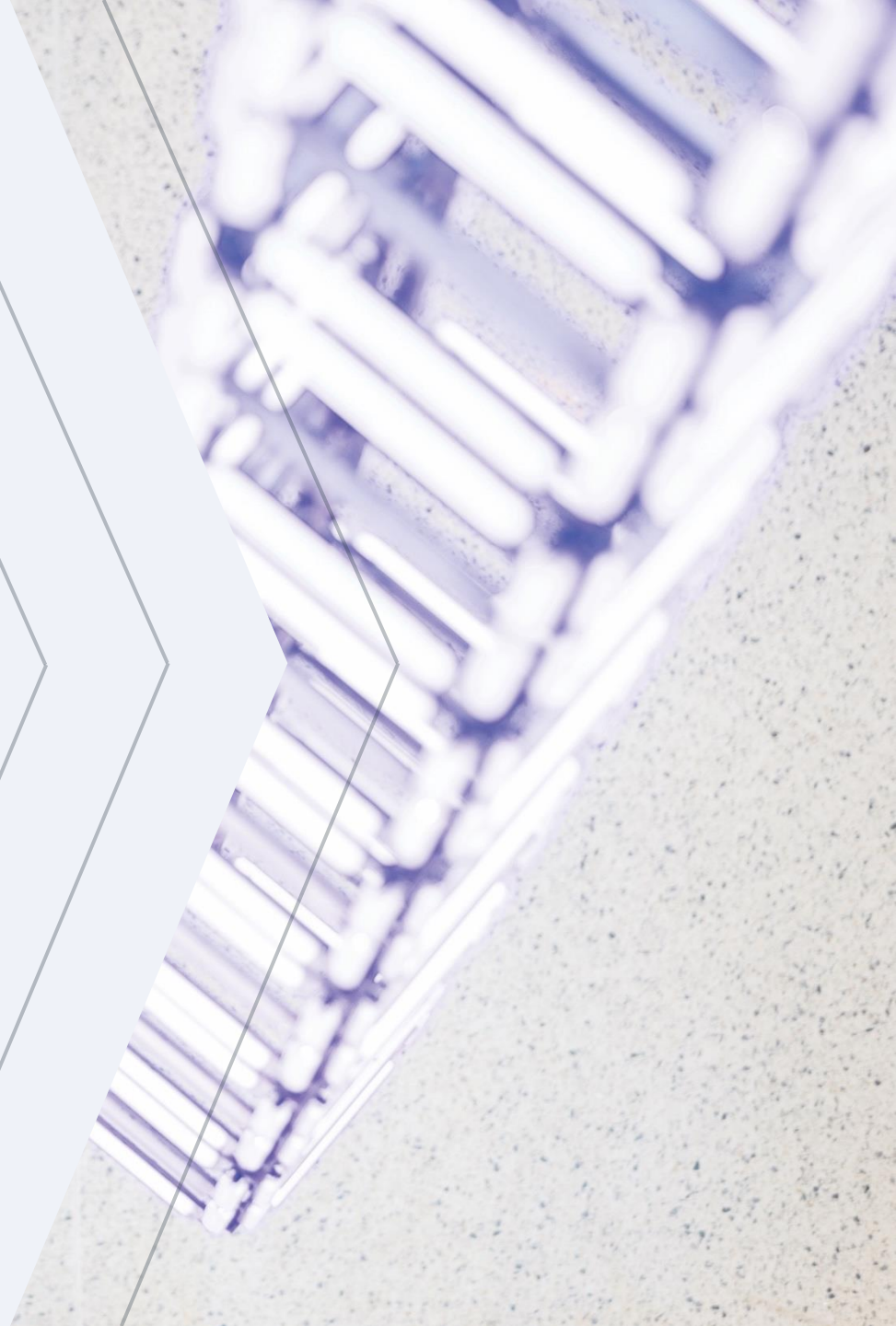
With some assistance from Claude

DS6021: Introduction to Predictive Modeling



Linear Regression Review

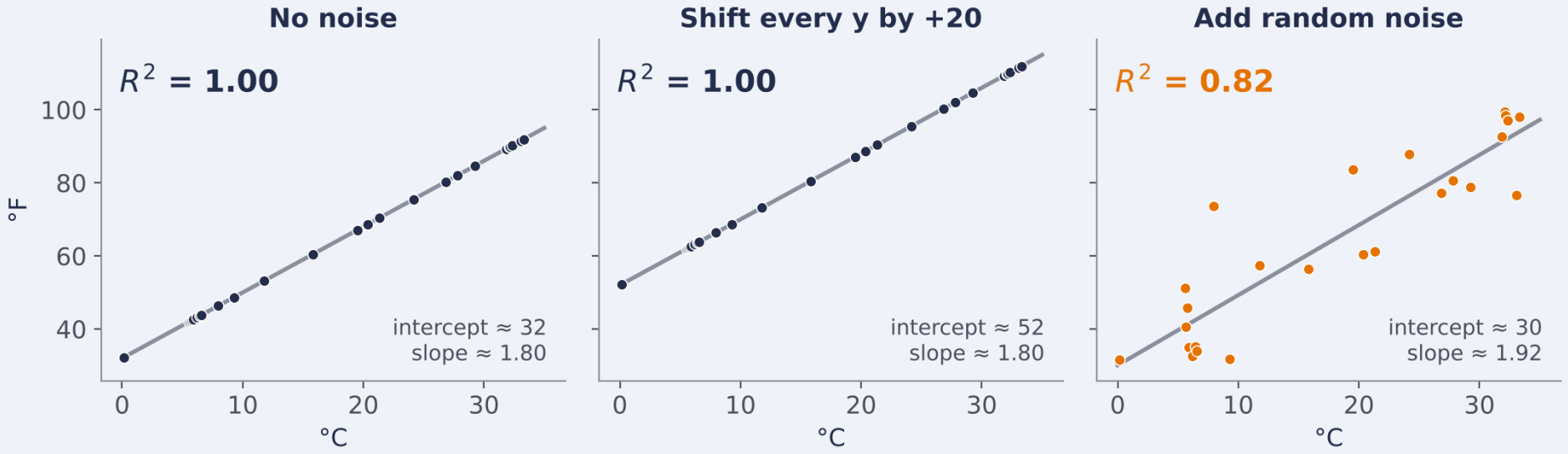
R^2, coefficient tables, VIF, and interactions



Review of R²

$$R^2 = \frac{TSS - RSS}{TSS} = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

TSS: spread of y around its mean
RSS: spread of y around the fitted line



No noise → R² = 1. Every point sits on the line, so RSS is zero. OLS recovers the exact rule: intercept 32, slope 1.8.

Shifting every y leaves R² unchanged. The line moves, but the scatter around it doesn't. Only noise around the line lowers R².

Interpreting a coefficient table

Example: A model predicts weekly grocery spend (in tens of dollars) from household size (people), income ($1,000s), and distance to the store (miles)

Term	Coef	P-value
Intercept	4.0	0.002
Household size	1.8	<0.001
Income	0.05	0.010
Distance	−0.30	0.210

Q1: Interpret the household size coefficient

Q2: Write the equation for the weekly grocery spend for the following household:

3 people, $80k income, 2 miles from the store



Interpreting a coefficient table

Example: A model predicts weekly grocery spend (in tens of dollars) from household size (people), income ($1,000s), and distance to the store (miles)

Term	Coef	P-value
Intercept	4.0	0.002
Household size	1.8	<0.001
Income	0.05	0.010
Distance	−0.30	0.210

Q1: Interpret the household size coefficient

"Each additional person is associated with **1.8 × $10 = $18** more weekly spend, on average, **holding income and distance fixed**."

- Units of X: **per one person**
- Units of Y: **convert tens of dollars**
- Other predictors held constant

Q2: Write the equation for the weekly grocery spend for the following household: :

3 people, $80k income, 2 miles from the store

$$\widehat{\text{Spend}} = 4.0 + 1.8(3) + 0.05(80) - 0.30(2)$$

= 12.8 tens of dollars
≈ $128 per week

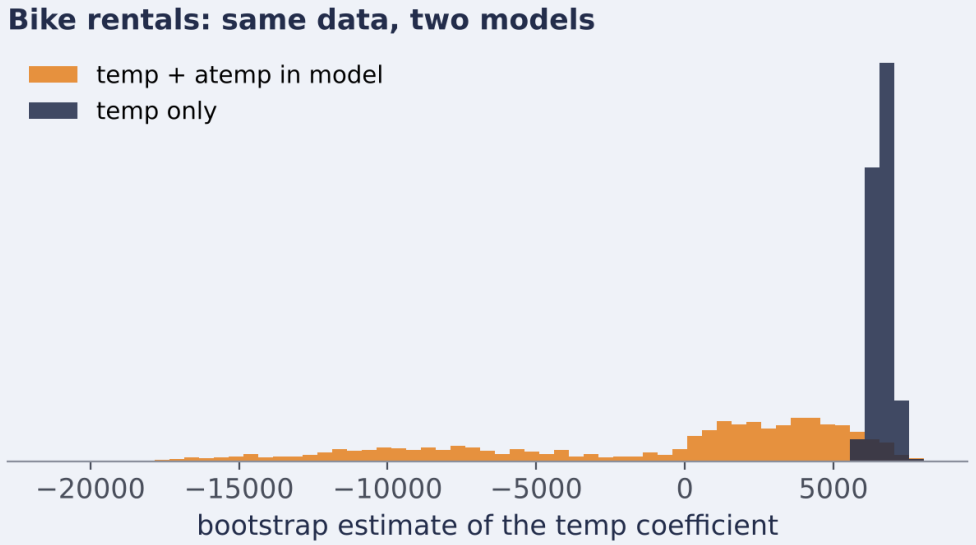


Variance Inflation Factor

$$VIF(\hat{\beta}_j) = \frac{1}{1 - R_j^2}$$

What does the VIF and the example plot on the right tell us?

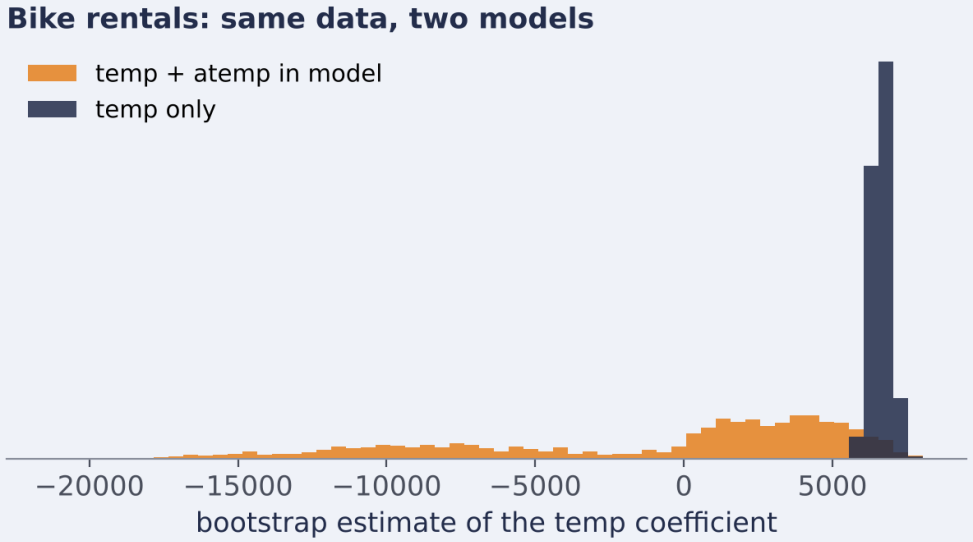
Does a high VIF mean drop the variable?



Variance Inflation Factor

$$VIF(\hat{\beta}_j) = \frac{1}{1 - R_j^2}$$

- **R_j^2** comes from regressing predictor j on all the other predictors. Rules of thumb:
  - VIF = 1: unrelated.
  - > 5 worth a look
  - > 10 likely significant multicollinearity.
- **High VIF → bigger standard errors** → wider confidence intervals and larger p-values. A useful predictor can look "not significant."
- VIF tells you X_j is predictable from the others, **not which ones**. Check correlations to find the partner.
- **It barely affects predictions.** Fitted values stay about the same.



Does a high VIF mean drop the variable? Not automatically. It depends on the goal.

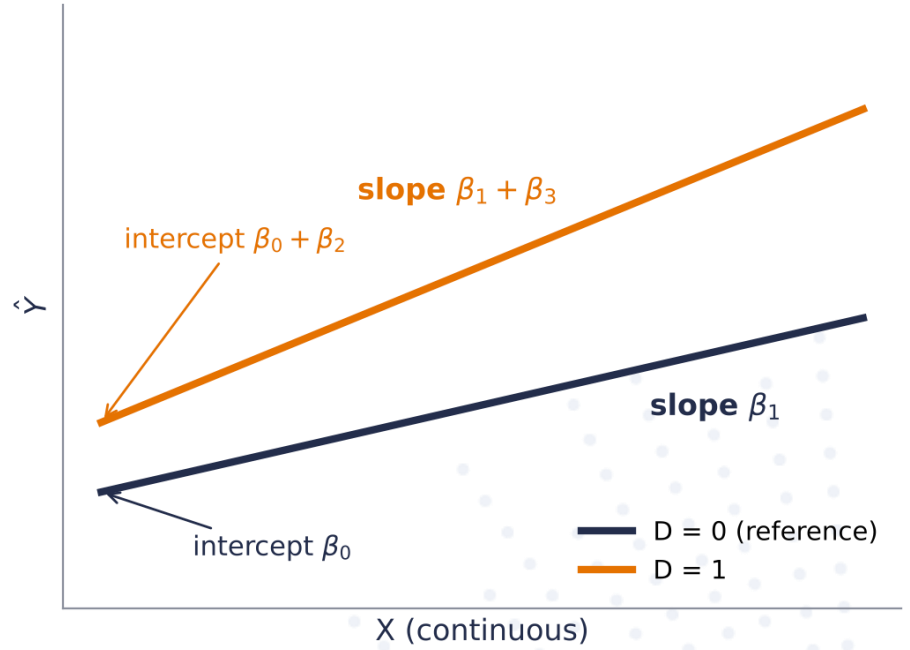
Inference: unstable coefficients are a problem → drop one, combine, or regularize.

Prediction: collinearity rarely hurts accuracy → let cross-validation decide.

Interaction Terms

$$Y = \beta_0 + \beta_1 X + \beta_2 D + \beta_3 (X \cdot D) + \epsilon$$

$$D = 0: \quad \hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X$$
$$D = 1: \quad \hat{Y} = (\hat{\beta}_0 + \hat{\beta}_2) + (\hat{\beta}_1 + \hat{\beta}_3) X$$



β_0 intercept for the reference group (D = 0)

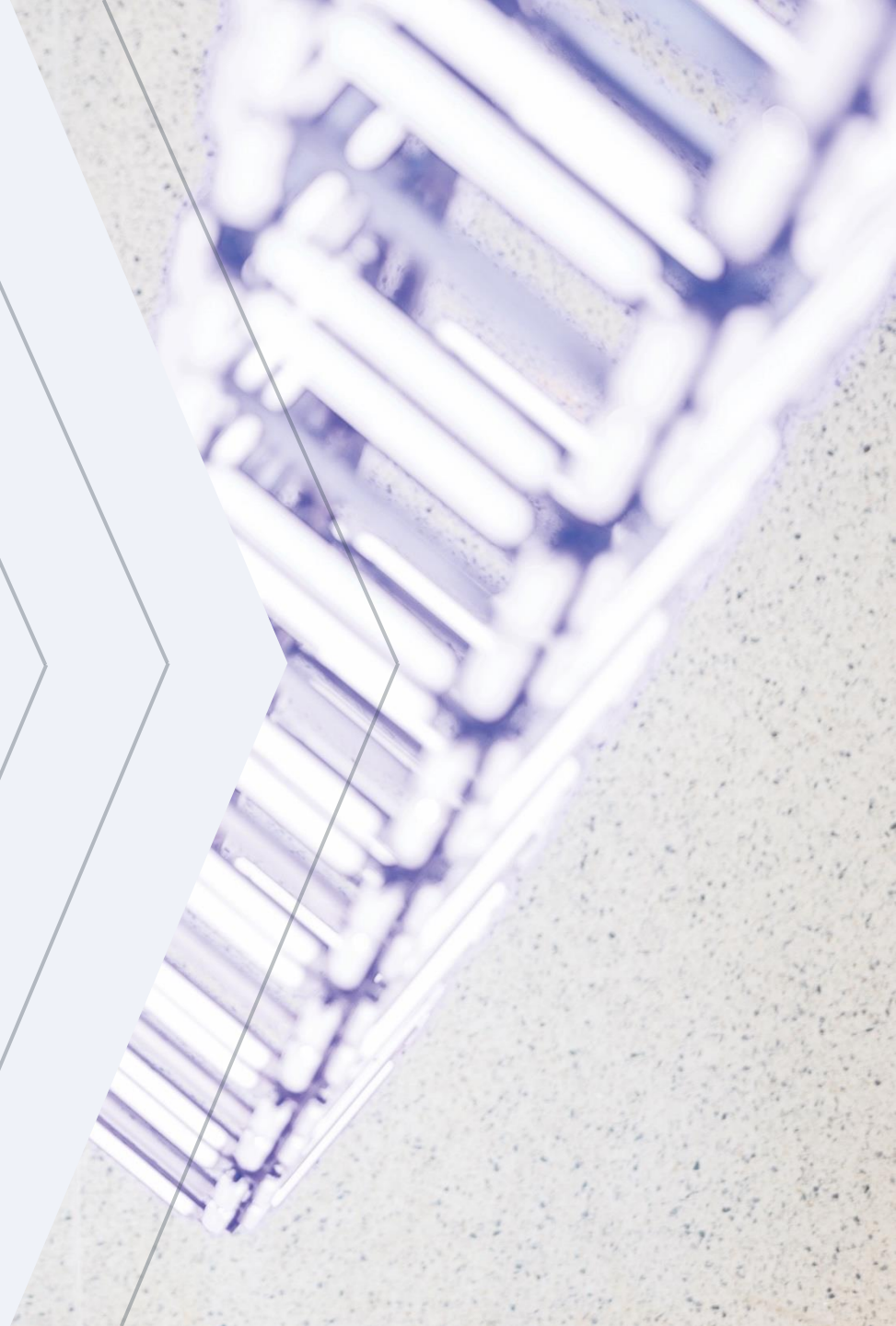
β_1 slope of X **for the reference group only**. It is not an "overall" effect.

β_2 difference in intercepts (D = 1 minus D = 0) at X = 0

β_3 difference in slopes: the extra effect of X when D = 1

Keep both main effects (X and D) whenever you include X·D.

Pipelines and Cross-Validation



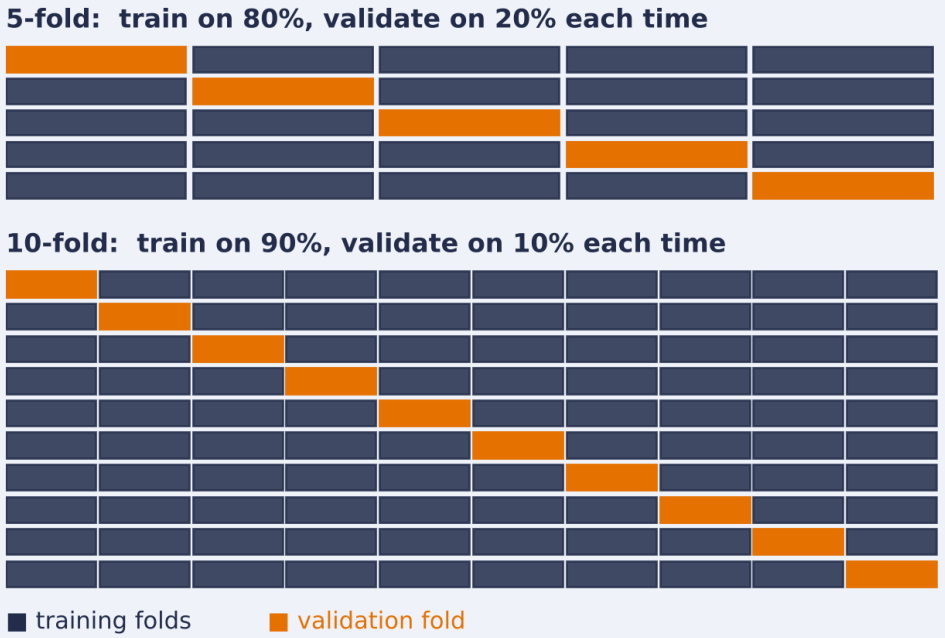
What a CV score estimates

$$\mathrm{CV}_{(k)} = \frac{1}{k}\sum_{i=1}^{k}\mathrm{MSE}_i$$

- Each MSE_i is computed on a fold the model **did not train on**.
- The average estimates **expected test error**: how well this modeling approach predicts new data.
- More stable than one train/test split, which depends on the luck of a single split.
- The **spread across folds** matters too: a big SD means the estimate is noisy.

Example: fold MSEs = 4.1, 3.8, 4.6, 4.0, 3.9

CV MSE = (4.1 + 3.8 + 4.6 + 4.0 + 3.9) / 5 = **4.08**



Impact of different values of k in k-fold CV

	5-fold	10-fold	LOOCV (k = n)
Training share per fit	80%	90%	$(n - 1)/n \approx 100\%$
Models fit	5	10	n
Bias of the error estimate	Most pessimistic	Smaller	Nearly unbiased
Variance of the estimate	Lower	Moderate	Can be high
Compute cost	Lowest	Moderate	Highest

How does this impact the bias/variance tradeoff? The final model trains on all n rows, but each CV model sees only $(k - 1)/k$ of them.

K=10 means each training run has 90% of the data vs 80% with k=5. More training data is closer to the final model, and will have a lower out-of-sample error estimate. Bias is error introduced by the model itself, and more data will reduce this error. Higher k increases the variance in the error estimate by a moderate amount because the training sets overlap more.

Good discussion in ISL 5.1.4

Inside a preprocessing pipeline

NUMERIC COLUMNS

StandardScaler

Learns the **mean and SD from training data**, then rescales so the column has mean 0, SD 1.

$$z = \frac{x - \bar{x}_{\text{train}}}{s_{\text{train}}}$$

CATEGORICAL COLUMNS

`OneHotEncoder(drop="first")`

A k-level category becomes **k − 1 indicator (0/1) columns**.

The dropped level is the **reference level**.

region = {North, South, East, West} → 3 columns: South, West (North = reference)

MODEL

LinearRegression

Fits coefficients on the transformed columns.

Interpret each coefficient in the **transformed** units.

Pipeline chains the steps in order and treats them as one model

Why is the procedure on the left incorrect?

✗ **Wrong**

```
X_scaled =
StandardScaler().fit_transform(X)

X_tr, X_te, y_tr, y_te = train_test_split(
    X_scaled, y)

model = LinearRegression().fit(X_tr, y_tr)
model.predict(X_te, y_te)
```

✓ **Correct**

```
X_tr, X_te, y_tr, y_te =
train_test_split(X, y)

pipe = make_pipeline(StandardScaler(),
                     LinearRegression())

pipe.fit(X_tr, y_tr)
pipe.predict(X_te, y_te)
```

Data leakage!

✗ **Leaky**

```
X_scaled = StandardScaler().fit_transform(X)
X_tr, X_te, y_tr, y_te = train_test_split(
    X_scaled, y)
model = LinearRegression().fit(X_tr, y_tr)
model.score(X_te, y_te)   # too optimistic
```

The scaler's mean and SD were computed with the test rows included.

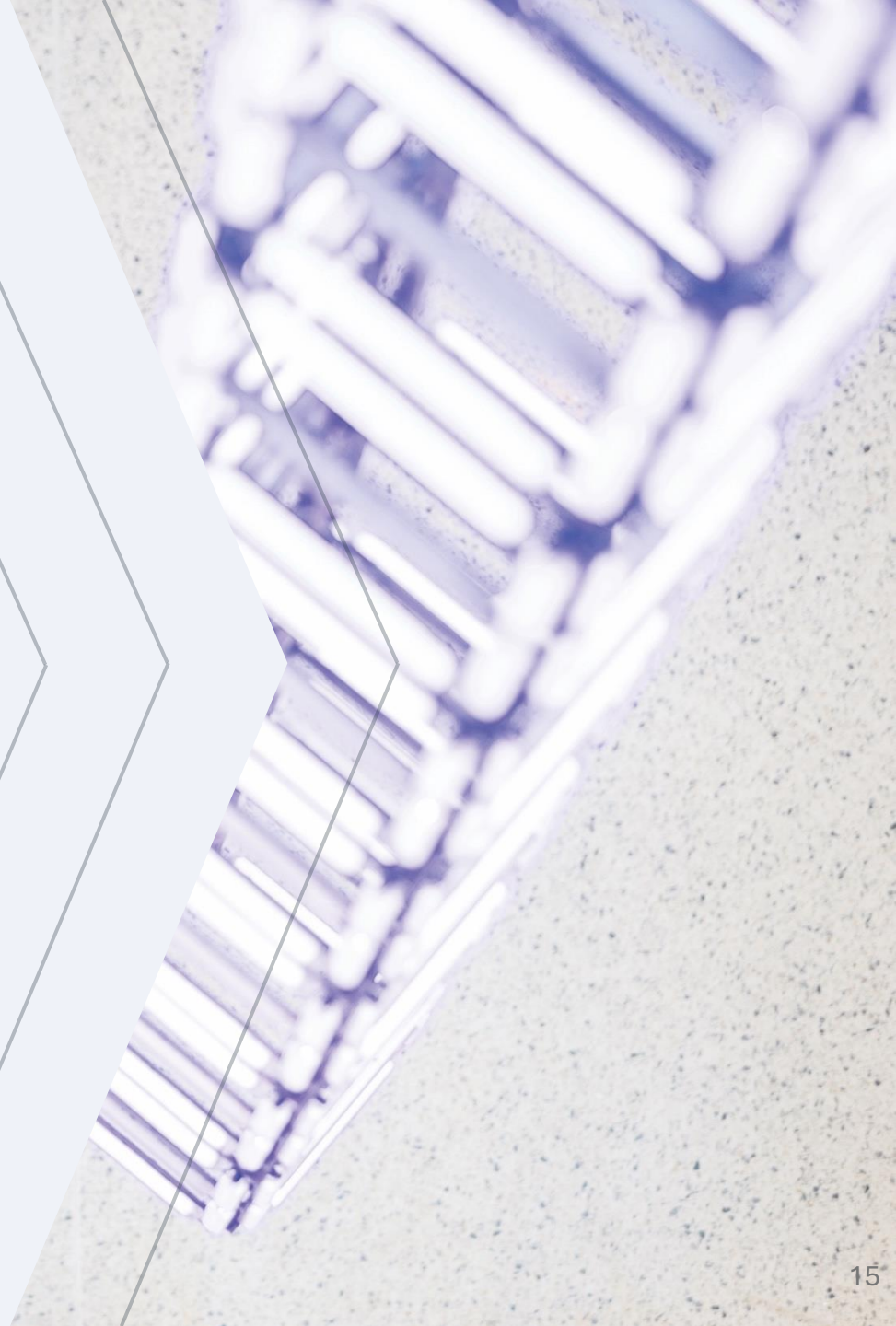
✓ **Correct**

```
X_tr, X_te, y_tr, y_te = train_test_split(X, y)
pipe = make_pipeline(StandardScaler(),
                     LinearRegression())
pipe.fit(X_tr, y_tr)      # mean/SD from train only
pipe.score(X_te, y_te)    # honest estimate
```

Split first. In cross-validation, the pipeline refits the scaler inside every fold.

- The procedure on the left includes test data in scaling (and any other preprocessing we might do)
- It will bias the out-of-sample test error since that data was included in the original scaling.
- You want any encoding and fitting to depend on training data only.

Regularization



Why regularization helps with many correlated features.

- In general, regularization introduces some bias into the model through the penalty term. Remember that the penalty term is a constraint.

- In ordinary least squares, lots of correlated predictors mean that many different combinations of coefficients fit almost equally well. This means that the coefficients are unstable and have high variance.

- When several fits are almost as good (due to the correlated features), the regularization penalty chooses the one with smallest coefficients, giving the most stable answer.

- When we regularize, the penalty shrinks coefficients toward 0, which helps keep the model from fitting random noise in the training data. There is more bias but a significant drop in variance.
    - Training error rises a little (due to increased bias) but the variance often decreases more than bias rises, which means that test error often drops.

Regularization

	Ridge (L2)	Lasso (L1)
Penalty	$\lambda \Sigma \beta_j^2$	$\lambda \Sigma \lvert\beta_j\rvert$
Coefficients	Shrink toward 0, never exactly 0	Some set exactly to 0
Feature selection	No: keeps every feature	Yes
Correlated group	Spreads weight across it	Tends to keep one

Both Ridge and Lasso regression will increase bias, reduce variance, and give more stable coefficient estimates, which generalize better to unseen data when the right λ is chosen by CV.

